

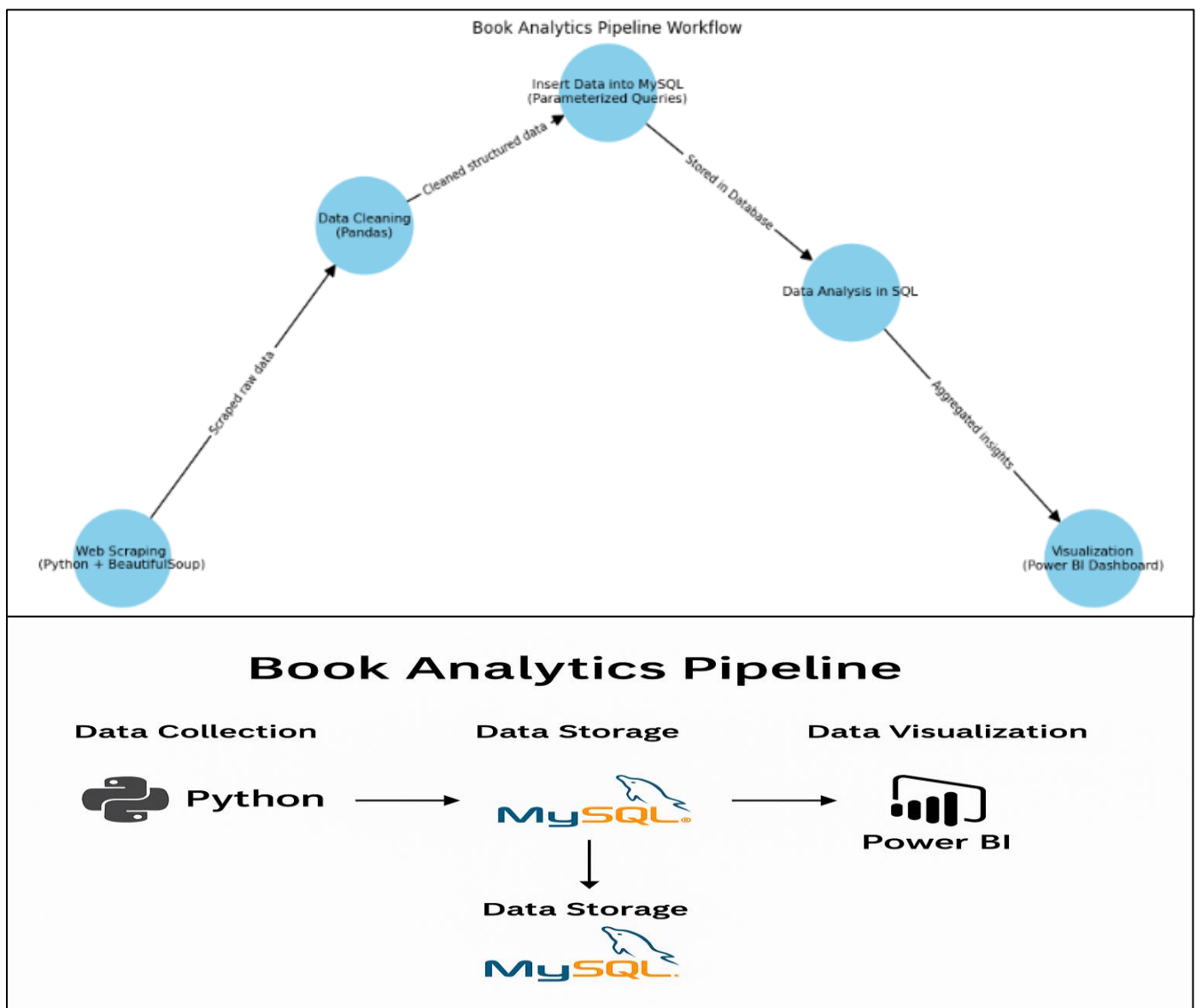
Book Analytics Pipeline using Python, MySQL & Power BI

From raw web data to actionable book insights

❖ Abstract:

- Scraped 1000+ book records from *Books to Scrape* using Python (BeautifulSoup) for automated data acquisition.
- Cleaned, structured, and normalized datasets using Pandas before storing them in MySQL for optimized querying.
- Developed an interactive Power BI dashboard to visualize book ratings, pricing, availability and categories.
- Enabled dynamic filtering, drill-down analysis and KPI monitoring for actionable business insights.
- Tools & Technologies Used: Python, Pandas, BeautifulSoup, MySQL, Power BI.

❖ Workflow Diagram:



❖ Data Collection (Python):

➤ This code imports the Python libraries needed for the project:

- requests – for sending HTTP requests to retrieve web pages.
- BeautifulSoup (bs4) – for parsing HTML and extracting book details.
- pandas – for storing and processing the scraped data in tabular form.
- urllib – for combining relative and absolute URLs to get complete product links.

```
[1]: import requests
      from bs4 import BeautifulSoup
      import pandas as pd
      from urllib.parse import urljoin
```

➤ Here, a loop runs through **50 pages** of the *Books to Scrape* website:

- requests.get() retrieves each page.
- BeautifulSoup parses the HTML to find all book entries.
- For each book, it extracts Title, Price, Availability, Rating and Product Link.
- Data is appended to a list (book_list) and a message confirms each page's success.

```
base_url = "http://books.toscrape.com/catalogue/page-{}.html"
book_list = []

for page in range(1, 51): # 1 to 50
    url = base_url.format(page)
    response = requests.get(url)
    if response.status_code != 200:
        print(f"Failed to retrieve page {page}")
        continue

    soup = BeautifulSoup(response.text, 'html.parser')
    books = soup.select('article.product_pod')

    for book in books:
        title = book.h3.a['title']
        price = book.select_one('.price_color').text.strip()
        availability = book.select_one('.availability').text.strip()
        rating_class = book.select_one('p.star-rating')['class'][1]
        link = urljoin(url, book.h3.a['href'])

        book_list.append({
            'Title': title,
            'Price': price,
            'Availability': availability,
            'Rating': rating_class,
            'Product_Link': link
        })

    print(f"✅ Scraped page {page}")
```

```
✅ Scraped page 1
✅ Scraped page 2
✅ Scraped page 3
✅ Scraped page 4
✅ Scraped page 5
✅ Scraped page 6
✅ Scraped page 7
✅ Scraped page 8
✅ Scraped page 9
✅ Scraped page 10
✅ Scraped page 11
✅ Scraped page 12
✅ Scraped page 13
```

➤ Converting Data to DataFrame:

- The scraped book data is loaded into a pandas DataFrame for easy analysis.
- The first view (head()) shows the top 5 records.
- The Product_Link column is removed because it's not required for further analysis.

```
[3]: df = pd.DataFrame(book_list)

[4]: print(df.head())
```

	Title	Price	Availability	Rating
0	A Light in the Attic	Â£51.77	In stock	Three
1	Tipping the Velvet	Â£53.74	In stock	One
2	Soumission	Â£50.10	In stock	One
3	Sharp Objects	Â£47.82	In stock	Four
4	Sapiens: A Brief History of Humankind	Â£54.23	In stock	Five

```
Product_Link
0 http://books.toscrape.com/catalogue/a-light-in-...
1 http://books.toscrape.com/catalogue/tipping-th...
2 http://books.toscrape.com/catalogue/soumission...
3 http://books.toscrape.com/catalogue/sharp-obje...
4 http://books.toscrape.com/catalogue/sapiens-a-...

[5]: df.drop('Product_Link', axis=1, inplace=True)

[6]: df.head()
```

	Title	Price	Availability	Rating
0	A Light in the Attic	Â£51.77	In stock	Three
1	Tipping the Velvet	Â£53.74	In stock	One
2	Soumission	Â£50.10	In stock	One
3	Sharp Objects	Â£47.82	In stock	Four
4	Sapiens: A Brief History of Humankind	Â£54.23	In stock	Five

➤ Data Cleaning & Formatting:

- Price column: Removes the currency symbol and converts it to a float for numerical analysis.
- Rating column: Maps text ratings (One–Five) to numeric values (1–5).
- Availability column: Converts “In stock” to “Yes” and others to “No” for clarity.

```
df['Price'] = df['Price'].str.replace('Â£', '').astype(float)

rating_map = {'One': 1, 'Two': 2, 'Three': 3, 'Four': 4, 'Five': 5}
df['Rating'] = df['Rating'].map(rating_map)

df['Availability'] = df['Availability'].apply(lambda x: 'Yes' if 'In stock' in x else 'No')

df
```

	Title	Price	Availability	Rating
0	A Light in the Attic	51.77	Yes	3
1	Tipping the Velvet	53.74	Yes	1
2	Soumission	50.10	Yes	1
3	Sharp Objects	47.82	Yes	4
4	Sapiens: A Brief History of Humankind	54.23	Yes	5
...	...	...	...	...
995	Alice in Wonderland (Alice's Adventures in Won...	55.53	Yes	1
996	Ajin: Demi-Human, Volume 1 (Ajin: Demi-Human #1)	57.06	Yes	4
997	A Spy's Devotion (The Regency Spies of London #1)	16.97	Yes	5
998	1st to Die (Women's Murder Club #1)	53.98	Yes	1
999	1,000 Places to See Before You Die	26.08	Yes	5

1000 rows x 4 columns

➤ MySQL Connector Installation & Database Connection:

- Installs required Python libraries: mysql-connector-python and pymysql.
- Connects to the MySQL database BookAnalyticsDB using login credentials.
- Creates a cursor object for executing SQL queries.

```
pip install mysql-connector-python
```

```
Collecting mysql-connector-pythonNote: you may need to restart the kernel to use updated packages.
```

```
  Downloading mysql_connector_python-9.4.0-cp313-cp313-win_amd64.whl.metadata (7.7 kB)
```

```
  Downloading mysql_connector_python-9.4.0-cp313-cp313-win_amd64.whl (16.4 MB)
```

```
----- 0.0/16.4 MB ? eta -:--:--
----- 2.1/16.4 MB 11.1 MB/s eta 0:00:02
----- 4.5/16.4 MB 11.1 MB/s eta 0:00:02
----- 6.8/16.4 MB 11.1 MB/s eta 0:00:01
----- 8.9/16.4 MB 11.1 MB/s eta 0:00:01
----- 11.5/16.4 MB 11.2 MB/s eta 0:00:01
----- 13.9/16.4 MB 11.3 MB/s eta 0:00:01
----- 15.2/16.4 MB 10.7 MB/s eta 0:00:01
----- 16.3/16.4 MB 10.2 MB/s eta 0:00:01
----- 16.4/16.4 MB 9.7 MB/s eta 0:00:00
```

```
Installing collected packages: mysql-connector-python
```

```
Successfully installed mysql-connector-python-9.4.0
```

```
pip install mysql-connector-python sqlalchemy pymysql
```

```
Requirement already satisfied: mysql-connector-python in d:\hardik\pythonuse\lib\site-packages (9.4.0)
```

```
Requirement already satisfied: sqlalchemy in d:\hardik\pythonuse\lib\site-packages (2.0.39)
```

```
Collecting pymysql
```

```
  Downloading PyMySQL-1.1.1-py3-none-any.whl.metadata (4.4 kB)
```

```
Requirement already satisfied: greenlet!=0.4.17 in d:\hardik\pythonuse\lib\site-packages (from sqlalchemy) (3.1.1)
```

```
Requirement already satisfied: typing-extensions>=4.6.0 in d:\hardik\pythonuse\lib\site-packages (from sqlalchemy) (4.14.1)
```

```
  Downloading PyMySQL-1.1.1-py3-none-any.whl (44 kB)
```

```
Installing collected packages: pymysql
```

```
Successfully installed pymysql-1.1.1
```

```
Note: you may need to restart the kernel to use updated packages.
```

```
import mysql.connector
```

```
import pandas as pd
```

```
conn = mysql.connector.connect(
    host='localhost',
    user='root',
    password='',
    database='BookAnalyticsDB')
cursor = conn.cursor()
```

➤ Parameterized Data Insertion into MySQL:

- Uses an INSERT INTO SQL query to insert book records into the MySQL Books table.
- Loops through each row in the DataFrame and inserts the values using parameterized queries (avoids SQL injection).
- Commits the transaction and closes the connection after successful insertion.

```
insert_query = """INSERT INTO Books (Title, Price, Availability, Rating)
VALUES (%s, %s, %s, %s)"""
```

```
for _, row in df.iterrows():
    data = (row['Title'], row['Price'], row['Availability'], row['Rating'])
    cursor.execute(insert_query, data)
```

```
conn.commit()
cursor.close()
conn.close()
print("All records inserted into MySQL successfully.")
```

```
All records inserted into MySQL successfully.
```

❖ Database Design (SQL):

- Creates a new database named BookAnalyticsDB and selects it for use.
- Database & Table Creation in MySQL. Defines a Books table with the following fields:
 - BookID (auto-increment primary key)
 - Title (book name, up to 255 characters)
 - Price (decimal value with two decimal places)
 - Availability (only "Yes" or "No" allowed via ENUM)
 - Rating (integer restricted between 1 and 5 using a CHECK constraint)

```
• CREATE DATABASE BookAnalyticsDB;
• USE BookAnalyticsDB;

• CREATE TABLE Books (
    BookID INT AUTO_INCREMENT PRIMARY KEY,
    Title VARCHAR(255),
    Price DECIMAL(6,2),
    Availability ENUM('Yes', 'No'),
    Rating INT CHECK (Rating BETWEEN 1 AND 5));
```

- Sample Data in MySQL Books Table:
- Executes a SELECT * FROM Books; query to view the table's current contents.
- Displays some book records with their IDs, titles, prices, availability status and ratings.
- Data confirms that:
 - Book IDs are sequential and unique.
 - All prices are valid decimal values.
 - Availability values follow the ENUM rule.
 - Ratings respect the 1–5 constraint.

11 • select * from books;

	BookID	Title	Price	Availability	Rating
▶	1	A Light in the Attic	51.77	Yes	3
	2	Tipping the Velvet	53.74	Yes	1
	3	Soumission	50.10	Yes	1
	4	Sharp Objects	47.82	Yes	4
	5	Sapiens: A Brief History of Humankind	54.23	Yes	5
	6	The Requiem Red	22.65	Yes	1
	7	The Dirty Little Secrets of Getting Your Dream Job	33.34	Yes	4
	8	The Coming Woman: A Novel Based on the Life ...	17.93	Yes	3
	9	The Boys in the Boat: Nine Americans and Their ...	22.60	Yes	4
	10	The Black Maria	52.15	Yes	1
	11	Starving Hearts (Triangular Trade Trilogy, #1)	13.99	Yes	2
	12	Shakespeare's Sonnets	20.66	Yes	4
	13	Set Me Free	17.46	Yes	5
	14	Scott Pilgrim's Precious Little Life (Scott Pilgrim #1)	52.29	Yes	5
	15	Rip it Up and Start Again	35.02	Yes	5
	16	Our Band Could Be Your Life: Scenes from the A...	57.25	Yes	3
	17	Olio	23.88	Yes	1
	18	Mesaerion: The Best Science Fiction Stories 180...	37.59	Yes	1

❖ Data Visualization (Power BI):

➤ Chart Details:

- Chart 1: Count of Books by Price Range:

Type: Column Chart

Purpose: Displays the distribution of books across different price ranges to identify pricing trends.

- Chart 2: Average Rating by Price Range:

Type: Column Chart

Purpose: Shows the average rating of books within each price range to correlate price with quality perception.

- Chart 3: Book Count by Price & Rating:

Type: Ribbon Chart

Purpose: Highlights the relationship between book price and rating, aiding in identifying high-rated price points.

- Chart 4: Rating Distribution:

Type: Donut Chart

Purpose: Shows the distribution of book ratings to quickly identify the most common rating levels.

- Chart 5: Book Count by Rating:

Type: Column Chart

Purpose: Displays the count of books for each rating level to understand rating frequency.

- Chart 6: Book Price by Rating Distribution:

Type: Donut Chart

Purpose: Illustrates the price distribution across different rating levels to analyze pricing trends by rating.

- Chart 7: Trend of Average Rating by Price Range:

Type: Line Chart

Purpose: Tracks the trend of average ratings across price ranges over time to identify patterns.

- Chart 8: KPI Cards (5-Star Rated Books, Total Books, Available Books, Average Price, Min Price, Max Price, Average of Rating):

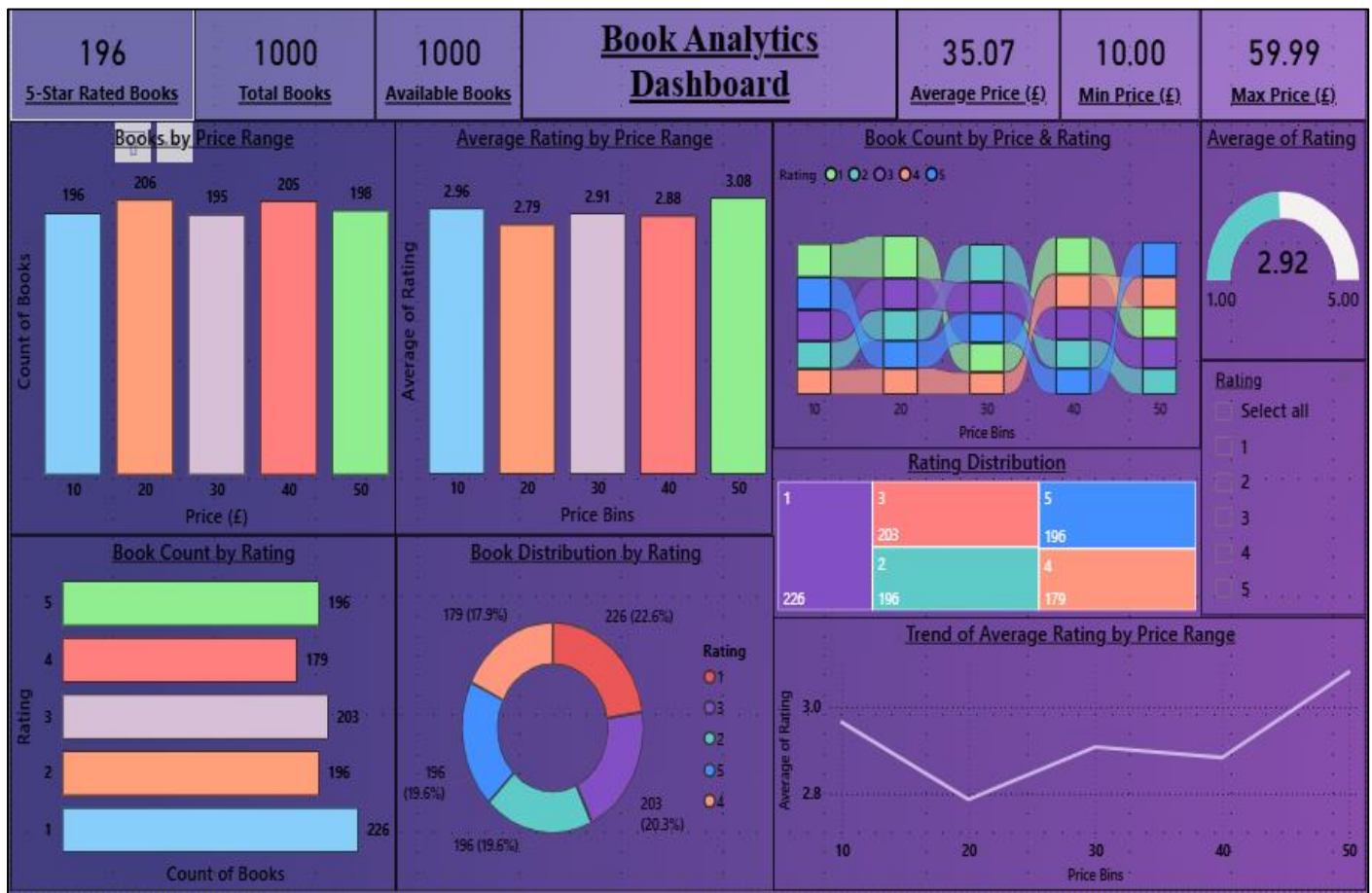
Type: KPI Cards

Purpose: Provides key metrics (e.g., total books, average price, rating) at a glance for quick insights.

- Chart 9: Rating Selector:

Type: KPI Cards

Purpose: Allows selection of specific ratings for filtering data.



❖ Key Features / Outcomes:

- **End-to-End Automated Data Pipeline:** The project automates the entire workflow, starting from scraping raw book data from the web, cleaning and structuring it with Python (Pandas), storing it in a MySQL database and finally visualizing it in Power BI without manual intervention.
- **Clean and Normalized Database:** The MySQL database is designed with a normalized schema that eliminates redundancy, improves data integrity and allows for faster query execution even as the dataset grows.
- **Interactive Power BI Dashboard:** Contains 15 visually rich charts, graphs, and KPI cards, enabling users to filter data dynamically with slicers and drill down into specific categories, time periods or other dimensions.
- **Genre and Category Analysis:** The dashboard enables detailed analysis of book distribution by category, helping identify popular genres, seasonal trends and category performance over time.
- **Time-Based Trend Analysis:** Visuals such as monthly book additions and rank changes allow tracking of trends, spikes or declines in different genres, which can help forecast future demand.
- **Business-Oriented Insights:** Designed to extract actionable insights, such as top-performing categories, average price trends and potential market gaps, aiding in informed decision-making for stakeholders.
- **Scalable and Future-Ready Design:** The architecture is built to easily incorporate additional data sources, scrape multiple websites or integrate real-time update mechanisms, ensuring adaptability to future needs.

❖ Conclusion:

- This project demonstrates how an automated data pipeline can seamlessly transform unstructured web data into actionable business insights. By combining Python-based scraping and cleaning, a clean and normalized MySQL database and an interactive Power BI dashboard, it delivers a complete analytics solution for book-related data. The system is inherently scalable, it can be expanded to scrape from multiple online bookstores, integrate real-time data feeds, or incorporate additional dimensions like customer reviews and sales figures. Such an approach is highly valuable for businesses, as it enables continuous tracking of book sales, monitoring of market trends and identification of emerging opportunities in the publishing industry, ultimately supporting more informed and data-driven decision-making.